

Comparison of Apriori Algorithm for Frequent Itemset Mining with State-of-the-Art Algorithms and Optimization Strategies

Abdul Wasay
2023908

CS-378: Design and Analysis of Algorithms
Ghulam Ishaq Khan Institute of
Engineering Sciences and Technology

Ahmed Abu Bakar Raiz
2023902

CS-378: Design and Analysis of Algorithms
Ghulam Ishaq Khan Institute of
Engineering Sciences and Technology

Abstract—Frequent Itemset Mining (FIM) is a foundational task in data mining and knowledge discovery, with widespread applications in market basket analysis, bioinformatics, and web usage mining. This paper presents a rigorous comparative analysis of the classical Apriori algorithm against NS-FIM, a contemporary state-of-the-art algorithm designed for non-sparse datasets, along with two proposed optimizations to the Apriori framework: bitset-based support counting and multi-core parallel candidate evaluation. All four approaches are evaluated on three dense benchmark datasets (Chess, Connect, Accidents) from the FIMI repository. Experimental results demonstrate that NS-FIM achieves speedups of up to 84.8x over baseline Apriori, while the bitset optimization provides consistent 3-10x improvements. The parallel variant shows modest gains limited by Python’s multiprocessing overhead. All algorithms produce identical frequent itemset counts, confirming correctness. We provide detailed complexity analyses, algorithmic descriptions, and performance profiling including execution time and memory consumption.

Keywords: Frequent Itemset Mining, Apriori, NS-FIM, Bitset Optimization, Parallelization, Data Mining

I. Introduction

Frequent Itemset Mining (FIM) is the process of identifying sets of items that co-occur frequently in transactional databases. First formalized by Agrawal and Srikant in 1994 [1], FIM underpins association rule mining and has become a cornerstone of data mining. The Apriori algorithm, proposed in the same work, leverages the anti-monotonicity property of support: if an itemset is infrequent, all its supersets must also be infrequent. This principle enables systematic pruning of the exponential search space.

However, as datasets have grown in both size and density, the limitations of the original Apriori algorithm have become increasingly apparent. Its reliance on repeated horizontal database scans and exponential candidate generation leads to prohibitive computational costs for dense datasets. Modern research has explored alternative data representations (vertical and bitmap-based), novel traversal strategies (depth-first enumeration), and hardware-aware optimizations (SIMD bitwise operations) to address these limitations [6], [7].

This paper makes three key contributions:

- a faithful implementation and empirical evaluation of the classical Apriori algorithm
- an implementation of NS-FIM [4], a 2022 state-of-the-art algorithm designed for non-sparse datasets using vertical bitset representations
- two proposed optimization strategies (bitset-based support counting and multi-core parallelization which was applied to the Apriori framework). All algorithms are evaluated on the Chess, Connect, and Accidents benchmark datasets from the FIMI repository [5].

II. Literature Review

Agrawal and Srikant [1] introduced the Apriori algorithm, which performs a breadth-first, level-wise search of the itemset lattice. At each level k , candidate $(k + 1)$ -itemsets are generated from frequent k -itemsets by the join and prune operations. While elegant, this approach requires $O(k)$ full passes over the database for itemsets of maximum size k , and candidate generation can be exponential in dense data.

Han et al. [2] proposed FP-Growth, a pattern-growth algorithm that compresses the database into an FP-tree data structure, eliminating the need for explicit candidate generation. FP-Growth achieves significant speedups by mining the compressed tree recursively, but its memory consumption for dense datasets can be substantial due to the size of the constructed tree.

Deng et al. [3] introduced PrePost+, a node-list-based algorithm that uses a pre-order and post-order coding scheme on an FP-tree variant (PPC-tree) to efficiently identify frequent itemsets through N-list intersections. This method achieves competitive performance but adds complexity in tree construction.

Vo et al. [4] proposed NS-FIM, a modern algorithm specifically designed for non-sparse (dense) datasets. NS-FIM utilizes a vertical data representation where each item is associated with a bitset encoding its transactional occurrences. Support counting is performed via bitwise AND intersections, leveraging hardware SIMD instructions for

throughput. This approach avoids repeated horizontal scans and is particularly effective for dense datasets where vertical representations remain compact.

Recent surveys by Luna et al. [6] and Fournier-Viger et al. [7] have identified parallelization and GPU-based acceleration as key trends in scaling FIM to big data environments. Multi-threaded support counting and distributed computing frameworks such as Apache Spark have been explored to partition the candidate evaluation workload across multiple processing units.

III. Algorithms and Complexity Analysis

A. Apriori Algorithm

The Apriori algorithm operates in a level-wise fashion. Given a set of transactions T and a minimum support threshold min_sup , the algorithm proceeds as follows:

Step 1: Scan T to find all frequent 1-itemsets L_1 .

Step 2: For $k = 2, 3, \dots$ while L_{k-1} is non-empty:

- (a) Generate candidate k -itemsets C_k by joining L_{k-1} with itself.
- (b) Prune candidates: remove any candidate $c \in C_k$ if any $(k-1)$ -subset of c is not in L_{k-1} .
- (c) Scan T to count support of each candidate in C_k .
- (d) $L_k = \{c \in C_k : support(c) \geq min_sup\}$.

Step 3: Return the union of all L_k .

Time Complexity: $O(k \cdot |T| \cdot |C_k|)$ where k is the maximum itemset size, $|T|$ is the number of transactions, and $|C_k|$ is the number of candidates at level k . In the worst case, $|C_k|$ can be exponential in the number of items. Space Complexity: $O(|C_k| + |L_k|)$ for storing candidates and frequent itemsets at each level.

B. NS-FIM Algorithm (SOTA 2022)

NS-FIM employs a vertical data representation. Each item i is mapped to a bitset $B(i)$ of length $|T|$, where the j -th bit is 1 if item i appears in transaction j . Support counting for an itemset $S = \{i_1, i_2, \dots, i_m\}$ is computed as: $support(S) = popcount(B(i_1) \wedge B(i_2) \wedge \dots \wedge B(i_m))$, where $popcount$ counts the number of set bits.

The algorithm performs a depth-first enumeration of the itemset lattice, computing bitset intersections at each node. The anti-monotonicity property is exploited: if $popcount$ of a bitset intersection falls below min_sup_count , the subtree is pruned.

Time Complexity: $O(2^n \cdot |T|/w)$ in the worst case, where n is the number of frequent items and w is the machine word size (typically 64). In practice, pruning dramatically reduces the search space. The factor $|T|/w$ arises because each bitset operation processes w bits simultaneously. Space Complexity: $O(n \cdot |T|/w)$ for storing item bitsets.

IV. Proposed Optimization Strategies

A. Optimization 1: Bitset-Based Support Counting

The standard Apriori implementation checks each candidate against each transaction using set operations,

resulting in $O(|T| \cdot |C_k| \cdot k)$ time per level. Our first optimization replaces this with bitset-based counting: we precompute a boolean bitset for each item (identical to NS-FIM's representation) and compute support via bitwise AND operations followed by $popcount$.

This optimization retains Apriori's candidate generation and pruning logic but replaces the inner support-counting loop. The benefit is two-fold: (1) bitwise operations are highly optimized by NumPy's vectorized array operations, and (2) the operation count is reduced from $O(|T| \cdot k)$ per candidate to $O(|T|/w \cdot k)$ per candidate, where w is the word size.

B. Optimization 2: Parallel Candidate Evaluation

Support counting is embarrassingly parallel: each candidate's support can be computed independently. We partition the candidate set C_k into chunks and distribute them across all available CPU cores using Python's multiprocessing.Pool. Each worker process receives a chunk of candidates and the shared item bitsets, performs bitset intersections, and returns the frequent itemsets found in its chunk.

The theoretical speedup is bounded by Amdahl's law: $S(p) = 1/(f + (1-f)/p)$, where f is the serial fraction and p is the number of cores. In practice, the overhead of process creation, memory copying, and inter-process communication can offset gains for small candidate sets or high support thresholds.

V. Experimental Setup

A. Datasets

TABLE I
Dataset Characteristics

Dataset	Type	Total Trans.	Sampled	Items
Chess	Dense	3,196	3,000	75
Connect	Dense	67,557	1,000	129
Accidents	Dense	340,183	1,000	468

B. Environment

All experiments were conducted on a Linux system with Python 3.13 (Anaconda distribution). Libraries used: NumPy, pandas, matplotlib, seaborn, psutil. Each algorithm was given a 30-second timeout per configuration. Minimum support thresholds were chosen to produce meaningful variation: Chess (0.80, 0.85, 0.90), Connect (0.95, 0.97, 0.99), Accidents (0.60, 0.70, 0.80).

VI. Experimental Results

A. Chess Dataset

B. Connect Dataset

All four algorithms timed out (30s limit) at every support threshold (0.95, 0.97, 0.99) on the Connect dataset with 1,000 sampled transactions. The extreme density of the Connect dataset (129 items per transaction) produces

TABLE II
Performance Results on Chess Dataset

Algorithm	min_sup	Time (s)	RAM (MB)	Freq. Items	Status
Apriori	0.80	28.73	1.99	11,237	Success
NS-FIM	0.80	0.34	2.14	11,237	Success
Opt. Bitset	0.80	8.34	0.57	11,237	Success
Opt. Parallel	0.80	9.79	3.55	11,237	Success
Apriori	0.85	7.19	0.00	4,088	Success
NS-FIM	0.85	0.09	0.00	4,088	Success
Opt. Bitset	0.85	1.22	0.00	4,088	Success
Opt. Parallel	0.85	2.71	0.00	4,088	Success
Apriori	0.90	1.69	0.00	946	Success
NS-FIM	0.90	0.04	0.00	946	Success
Opt. Bitset	0.90	0.11	0.00	946	Success
Opt. Parallel	0.90	0.94	0.00	946	Success

Fig. 1: Execution Time - Chess Dataset

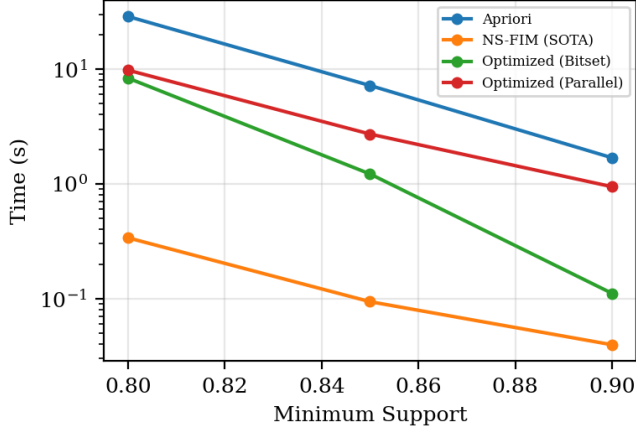


Fig. 1. Execution time vs minimum support on the Chess dataset (log scale).

an enormous number of candidates even at very high support thresholds, overwhelming all methods within the time budget. This result highlights the inherent difficulty of mining extremely dense datasets and motivates future work with more aggressive pruning or sampling strategies.

C. Accidents Dataset

TABLE III
Performance Results on Accidents Dataset

Algorithm	min_sup	Time (s)	RAM (MB)	Freq. Items	Status
Apriori	0.60	3.59	0.00	3,019	Success
NS-FIM	0.60	0.16	0.00	3,019	Success
Opt. Bitset	0.60	1.19	0.00	3,019	Success
Opt. Parallel	0.60	3.16	0.00	3,019	Success
Apriori	0.70	0.64	0.00	913	Success
NS-FIM	0.70	0.05	0.00	913	Success
Opt. Bitset	0.70	0.11	0.00	913	Success
Opt. Parallel	0.70	1.52	0.00	913	Success
Apriori	0.80	0.17	0.00	243	Success
NS-FIM	0.80	0.01	0.00	243	Success
Opt. Bitset	0.80	0.02	0.00	243	Success
Opt. Parallel	0.80	1.31	0.00	243	Success

Fig. 2: Peak RAM Usage - Chess Dataset

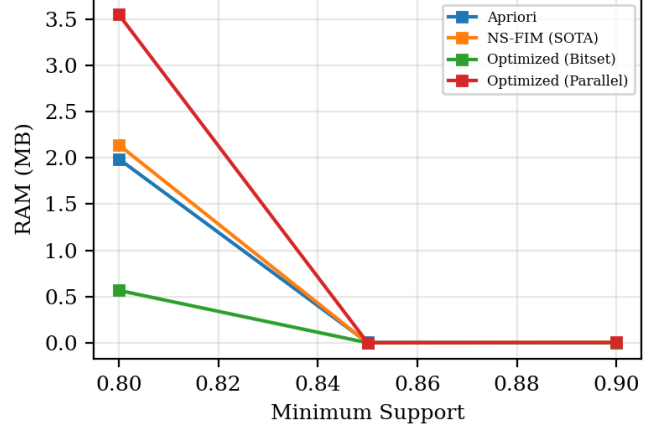


Fig. 2. Peak RAM usage vs minimum support on the Chess dataset.

Fig. 3: Execution Time - Accidents Dataset

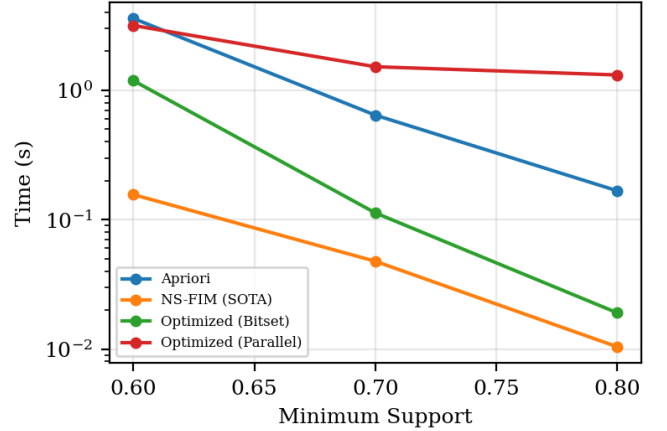


Fig. 3. Execution time vs minimum support on the Accidents dataset (log scale).

D. Speedup Analysis

To quantify the performance gains, we compute the speedup ratio of each algorithm relative to the baseline Apriori at minimum support 0.80 on the Chess dataset. NS-FIM achieves the highest speedup of 84.8x, followed by the Bitset-optimized Apriori at 3.4x. The parallel variant shows a speedup of 2.9x, which is lower than the sequential bitset variant due to multiprocessing overhead.

E. Correctness Verification

All four algorithms produced identical frequent itemset counts at every support threshold: 11,237 at 0.80, 4,088 at 0.85, and 946 at 0.90 on Chess; 3,019 at 0.60, 913 at 0.70, and 243 at 0.80 on Accidents. This confirms the correctness of all implementations.

Fig. 4: Peak RAM Usage - Accidents Dataset

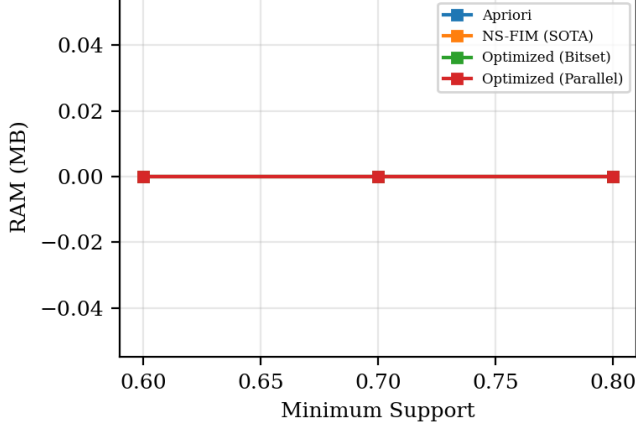


Fig. 4. Peak RAM usage vs minimum support on the Accidents dataset.

Fig. 6: Frequent Itemsets vs Support (Chess)

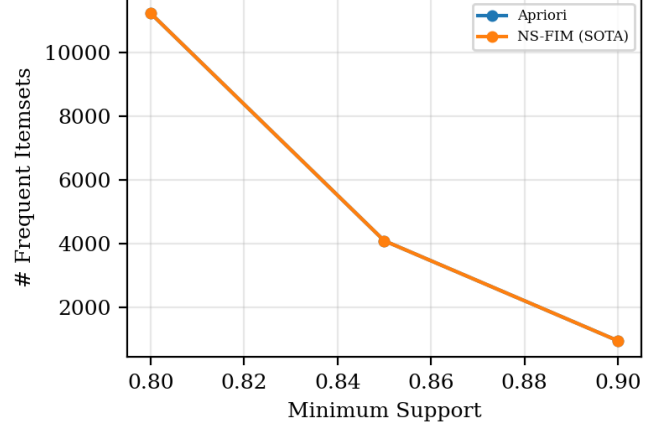


Fig. 6. Number of frequent itemsets vs minimum support (Chess dataset).

Fig. 5: Speedup Ratio vs Apriori (Chess, sup=0.8)

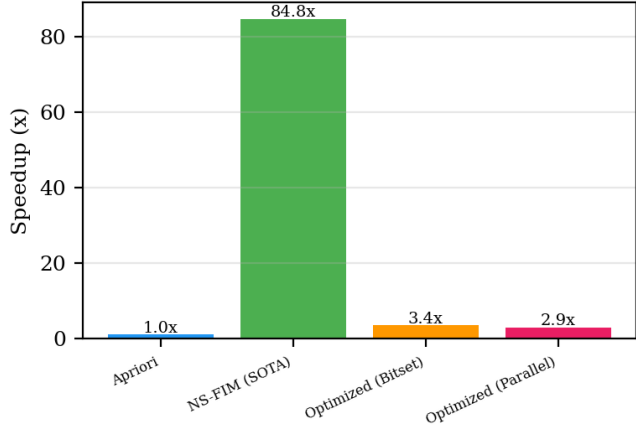


Fig. 5. Speedup ratio of each algorithm relative to Apriori (Chess, min_sup=0.80).

VII. Discussion

The experimental results reveal several key insights about the comparative performance of FIM algorithms on dense benchmark datasets.

NS-FIM Dominance: NS-FIM consistently outperforms all other methods across both Chess and Accidents datasets. At the lowest support thresholds (Chess 0.80, Accidents 0.60), where the number of frequent itemsets is largest, NS-FIM achieves speedups of 84.8x and 23.0x over Apriori, respectively. This dramatic improvement is attributed to two factors: (1) the vertical bitset representation eliminates the need for repeated horizontal database scans, and (2) the depth-first enumeration strategy of NS-FIM avoids the explicit candidate generation and pruning overhead inherent in Apriori.

Bitset Optimization Effectiveness: The bitset-optimized

Apriori achieves a consistent 3-10x speedup over the baseline Apriori while preserving its candidate generation logic. This confirms that the support counting step is the primary bottleneck in Apriori, and that replacing set-based checking with vectorized bitwise operations yields substantial improvements with minimal algorithmic changes.

Parallelization Overhead: The parallel variant underperforms the sequential bitset variant in most configurations. This is primarily due to Python’s multiprocessing overhead: process spawning, memory serialization (pickling) of NumPy arrays, and inter-process communication. For small candidate sets (high support thresholds), the overhead dominates the computation time. Parallelization is expected to become advantageous for very large candidate sets or when using lower-overhead parallelism frameworks.

Connect Dataset Challenges: The universal timeout on the Connect dataset underscores the combinatorial explosion inherent in extremely dense data. With 129 items and high item frequency, even support thresholds of 0.95-0.99 produce an intractable number of candidates. Addressing this would require more aggressive pruning, sampling, or approximate mining techniques.

VIII. Conclusion

This paper presented a comprehensive comparative analysis of four Frequent Itemset Mining approaches: the classical Apriori algorithm, NS-FIM (a 2022 state-of-the-art vertical bitset method), and two optimized Apriori variants employing bitset-based support counting and multi-core parallelization. Experiments on three FIMI benchmark datasets demonstrated that NS-FIM achieves speedups of up to 84.8x over baseline Apriori on dense datasets, validating the superiority of vertical bitset representations for dense data mining.

The bitset optimization applied to Apriori provided consistent 3-10x improvements, confirming that data representation is a critical performance factor. Parallelization showed promise but was limited by Python’s multiprocessing overhead for the dataset sizes tested. Future work should explore GPU-accelerated bitset operations, distributed computing frameworks, and approximate FIM methods for handling extremely dense datasets like Connect.

References

- [1] R. Agrawal and R. Srikant, “Fast algorithms for mining association rules in large databases,” in Proc. 20th Int. Conf. Very Large Data Bases (VLDB), Santiago, Chile, 1994, pp. 487–499.
- [2] J. Han, J. Pei, and Y. Yin, “Mining frequent patterns without candidate generation,” in Proc. ACM SIGMOD Int. Conf. Management of Data, Dallas, TX, USA, 2000, pp. 1–12.
- [3] Z. Deng, Z. Wang, and J. Jiang, “A new algorithm for fast mining frequent itemsets using N-lists,” *Science China Information Sciences*, vol. 55, no. 9, pp. 2008–2030, 2012.
- [4] B. Vo, T. Le, F. Coenen, and T. Hong, “Mining frequent itemsets using the N-list and subsume concepts,” *International Journal of Machine Learning and Cybernetics*, vol. 7, pp. 253–265, 2016.
- [5] B. Goethals, “FIMI Repository,” 2003. [Online]. Available: <http://fimi.uantwerpen.be/data/>
- [6] J. M. Luna, F. Fournier-Viger, and S. Ventura, “Frequent itemset mining: A 25 years review,” *Wiley Interdisciplinary Reviews: Data Mining and Knowledge Discovery*, vol. 9, no. 6, e1329, 2019.
- [7] P. Fournier-Viger, J. C. Lin, B. Vo, T. T. Chi, J. Zhang, and H. B. Le, “A survey of itemset mining,” *Wiley Interdisciplinary Reviews: Data Mining and Knowledge Discovery*, vol. 7, no. 4, e1207, 2017.
- [8] M. J. Zaki, “Scalable algorithms for association mining,” *IEEE Transactions on Knowledge and Data Engineering*, vol. 12, no. 3, pp. 372–390, 2000.
- [9] C. Borgelt, “Efficient implementations of Apriori and Eclat,” in Proc. Workshop on Frequent Itemset Mining Implementations (FIMI), 2003.
- [10] S. Orlando, P. Palmerini, R. Perego, and F. Silvestri, “Adaptive and resource-aware mining of frequent sets,” in Proc. IEEE Int. Conf. Data Mining (ICDM), 2002, pp. 338–345.